

Taller Formativo

Arquitectura de Microservicios

Notación C4 Nivel 3

Sistema de Gestión de Solicitudes
de Soporte Técnico de Internet

Equipo de trabajo:

- Cando Moreno Robinson Rodrigo — Arquitecto Líder
- Luna Mora Ernesto Gregory — Diseñador de Servicios
- Pacheco Cárdenas Cristhian Daniel — Responsable de Documentación

Docente: Gleiston Cicerón Guerrero Ulloa

Período Académico: Séptimo Semestre

Fecha de Entrega: 3 de Junio 2026

Índice

1. Descripción del sistema propuesto	2
1.1. Perfiles de usuario	2
2. Arquitectura de microservicios — C4 Nivel 3	2
2.1. Principios de diseño aplicados	2
2.2. Diagrama de componentes — C4 Nivel 3	3
3. Descripción de microservicios	3
3.1. Microservicio Cliente	3
3.1.1. Componentes internos	4
3.2. API Gateway	4
3.3. Auth Service	4
3.4. Ticket Service	4
3.5. Notif. Service	4
4. Canales de comunicación y justificación tecnológica	5
4.1. REST (síncrono)	5
4.2. gRPC (síncrono)	5
4.3. RabbitMQ (asíncrono)	5
5. Repositorio en GitHub	5
6. Referencias bibliográficas	6

1. Descripción del sistema propuesto

El sistema propuesto es una plataforma web orientada a la gestión estructurada de solicitudes de soporte técnico relacionadas con problemas en el servicio de Internet, dentro de entornos organizacionales que requieren trazabilidad, seguridad y eficiencia en la atención.

1.1. Perfiles de usuario

Cuadro 1: Roles del sistema y sus responsabilidades

Rol	Responsabilidades
Cliente	Registra solicitudes mediante formulario web, adjunta evidencias (PNG, JPG, PDF) y realiza seguimiento del estado de sus casos.
Técnico	Accede únicamente a las solicitudes asignadas; actualiza el estado (<i>Pendiente</i> , <i>Resuelta</i>) y registra acciones con marca de fecha, hora y usuario.
Administrador	Supervisa todas las solicitudes, asigna técnicos manualmente y tiene acceso completo a registros de auditoría y seguridad.
Superusuario	Gestión total de usuarios y roles: crea, modifica y elimina cuentas, asigna perfiles.

2. Arquitectura de microservicios — C4 Nivel 3

2.1. Principios de diseño aplicados

- **SRP** (*Single Responsibility Principle*): cada microservicio tiene exactamente una razón para cambiar.
- **Database-per-service**: cada servicio posee su propio almacenamiento; ningún servicio accede directamente a la base de datos de otro.
- **API Gateway** como único punto de entrada externo, responsable de enrutamiento, autenticación delegada y limitación de tasa.
- **Comunicación síncrona** (REST/gRPC) para operaciones que requieren respuesta inmediata; **comunicación asíncrona** (RabbitMQ) para desacoplar productores de consumidores en eventos de dominio.

2.2. Diagrama de componentes — C4 Nivel 3

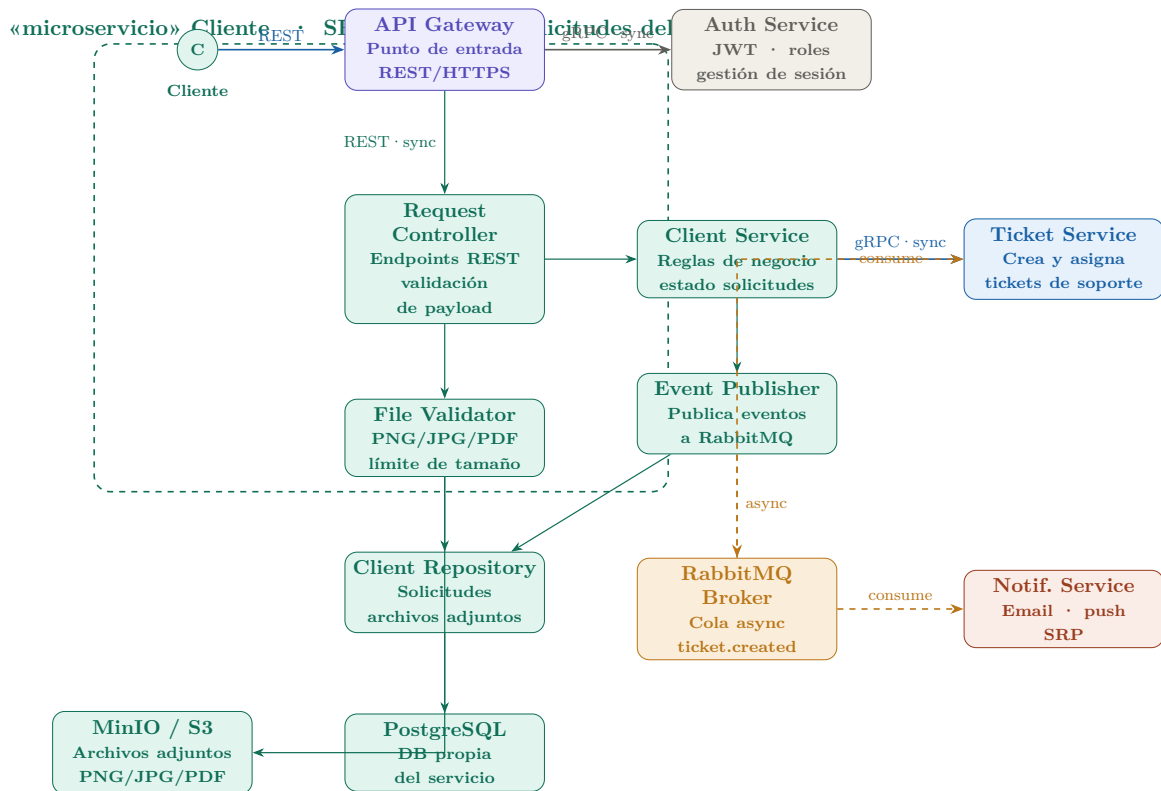


Figura 1: Diagrama C4 Nivel 3 — Microservicio Cliente del sistema de soporte técnico.

3. Descripción de microservicios

3.1. Microservicio Cliente

Responsabilidad única (SRP): Gestionar el ciclo de vida de las solicitudes de soporte enviadas por el usuario cliente: registro, validación de archivos, consulta de estado y emisión del evento de dominio `ticket.created`.

3.1.1. Componentes internos

Cuadro 2: Componentes del Microservicio Cliente

Componente	Descripción
Request Controller	Expone los endpoints REST (<code>POST /solicitudes</code> , <code>GET /solicitudes/{id}</code>). Valida el payload entrante y delega al servicio de negocio.
Client Service	Orquesta la lógica de negocio: crea, consulta y actualiza solicitudes. Coordina la validación de archivos y la publicación de eventos.
File Validator	Verifica el tipo MIME real (magic bytes), la extensión permitida (PNG, JPG, PDF) y el tamaño máximo configurable antes de delegar el almacenamiento.
Event Publisher	Serializa y publica el evento de dominio <code>ticket.created</code> en la cola de RabbitMQ. Implementa el patrón <i>outbox transactional</i> para garantizar entrega al menos una vez.
Client Repository	Capa de acceso a datos sobre PostgreSQL (propiedad exclusiva del servicio). Gestiona solicitudes y metadatos de archivos.

3.2. API Gateway

Punto de entrada único para todos los clientes externos. Responsable de: enrutamiento de peticiones, validación de token JWT (delegando al **Auth Service** vía gRPC), limitación de tasa (*rate limiting*) y terminación TLS.

3.3. Auth Service

Gestión de autenticación y autorización: emisión y validación de tokens JWT, control de roles (Cliente, Técnico, Administrador, Superusuario) y gestión de sesiones. Base de datos propia.

3.4. Ticket Service

Recibe el evento `ticket.created` de RabbitMQ y persiste el ticket en su propia base de datos. También responde a consultas síncronas del **Client Service** mediante gRPC para obtener el estado actual de un ticket.

3.5. Notif. Service

Consume el evento `ticket.created` y envía notificaciones al cliente (correo electrónico, push) confirmando la recepción de su solicitud.

4. Canales de comunicación y justificación tecnológica

4.1. REST (síncrono)

- **Uso:** comunicación entre el navegador del cliente y el API Gateway; desde el Gateway hacia el `Request Controller`.
- **Justificación:** estándar ampliamente adoptado, soporte nativo de adjuntos `multipart/form-data`, sencillo de consumir desde cualquier frontend web sin dependencias adicionales.

4.2. gRPC (síncrono)

- **Uso:** API Gateway → Auth Service; Client Service → Ticket Service.
- **Justificación:** baja latencia gracias a Protocol Buffers (binario) y HTTP/2. El contrato estricto mediante archivos `.proto` reduce errores de integración entre servicios internos.

4.3. RabbitMQ (asíncrono)

- **Uso:** Event Publisher → cola `ticket.created` → Ticket Service, Notif. Service.
- **Justificación frente a Kafka:** RabbitMQ es más apropiado para colas de trabajo con enrutamiento flexible (exchanges y routing keys), menor latencia en mensajes individuales y menor complejidad operacional para este dominio. Kafka sería preferible si se requiriera retención de logs y streaming de alto volumen.
- **Garantías:** entrega persistida, `acknowledge manual`, patrón *outbox transaccional* en el publicador.

5. Repositorio en GitHub

El Código en LaTeX de la práctica se encuentra disponible en: <https://github.com/CristhianP03/Actividades-Grupales-App-Distribuidas.git>

6. Referencias bibliográficas

Referencias

- [1] D. Taibi, V. Lenarduzzi y C. Pahl, “Processes, Motivations, and Issues for Migrating to Microservices Architectures: An Empirical Investigation,” *IEEE Cloud Computing*, vol. 4, n.º 5, pp. 22–32, 2017. DOI: 10.1109/MCC.2017.4250931
- [2] N. Dragoni *et al.*, “Microservices: Yesterday, Today, and Tomorrow,” en *Present and Ulterior Software Engineering*, pp. 195–216, Springer, 2017. DOI: 10.1007/978-3-319-67425-4_12
- [3] T. Cerny, M. J. Donahoo y M. Trnka, “Contextual Understanding of Microservice Architecture,” *ACM SIGAPP Applied Computing Review*, vol. 17, n.º 4, pp. 29–45, 2018. DOI: 10.1145/3183628.3183631
- [4] N. Alshuqayran, N. Ali y R. Evans, “A Systematic Mapping Study in Microservice Architecture,” en *Proc. IEEE SOCA 2016*, pp. 44–51, 2016. DOI: 10.1109/SOCA.2016.15
- [5] C. Richardson, *Microservices Patterns*. Manning Publications, 2018. ISBN: 978-1617294549.
- [6] F. Montesi y J. Weber, “Circuit Breakers, Discovery, and API Gateways in Microservices,” *arXiv:1609.05830*, 2016. Disponible en: <https://arxiv.org/abs/1609.05830>